

System Verilog for Design

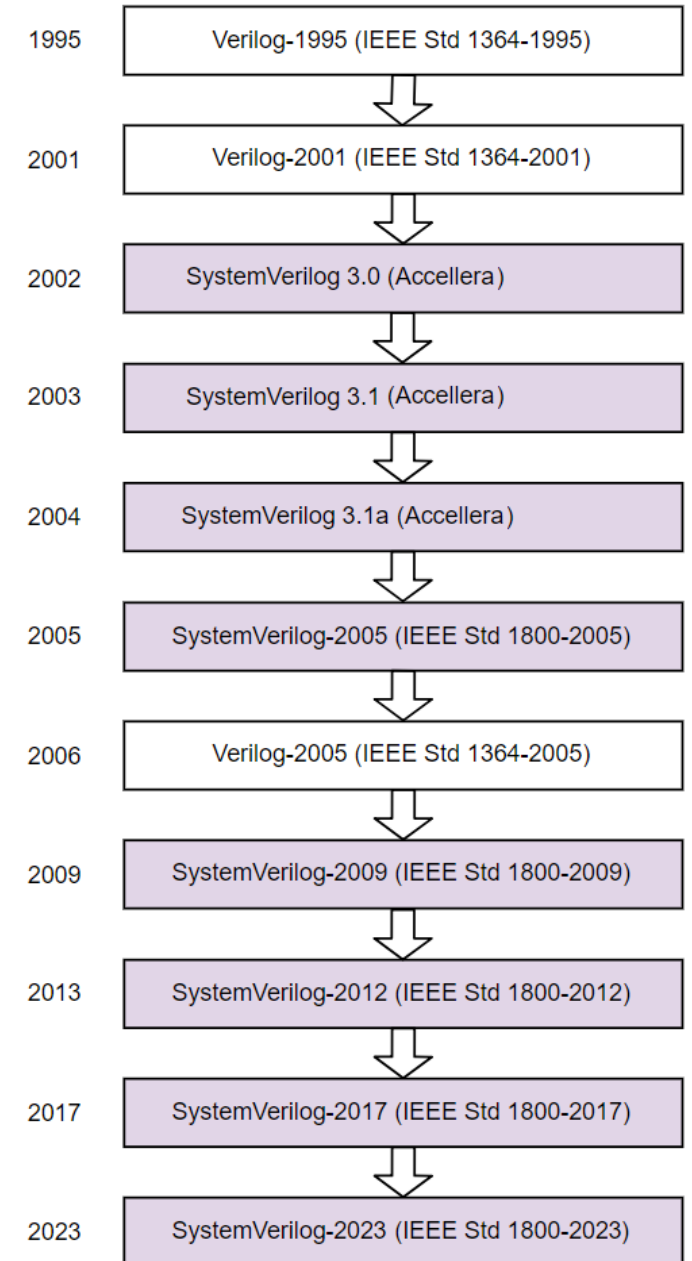
Transitioning from Verilog to SystemVerilog

SystemVerilog

Overview and Datatypes

System Verilog Overview

- System Verilog is a unified hardware design and verification language based on the Accellera System Verilog 3.1a
- It is a set of extensions to Verilog-2001, including the following:
 - Added data types and relaxation of rules on existing data types
 - Higher abstraction-level modeling features
 - Language enhancements for synthesis
 - Interfaces to model communication between design blocks
 - Language features to enable verification methodologies
 - Assertions, constrained randomization, functional coverage
 - Lightweight interface to C/C++ programs
- Aim
 - To define a concise, unified language for **design and verification** using a single simulation tool
 - Assuming tool support for enhanced testbench and verification features



Quick Reference Guide: SystemVerilog Language Extensions



							SystemVerilog IEEE 1800		
Literals time string pattern	Arrays packed unpacked dynamic associative queue	Classes new static this/super extends protected cast local	Statements unique priority do while foreach return break continue final iff	Processes always_comb always_latch always_ff join_any join_none wait fork	Random rand/randc constraint randomize() solve before dist with rand_mode constraint_mode randcase randsequence	Clocking clocking ##N	Coverage covergroup coverpoint cross wildcard sequence bins illegal_bins	Interfaces interface virtual modport export import	
Types logic bit byte shortint int longint shortreal void	Declarations const alias type var	Operators assignment wild equality tagged overloading		Subroutines static automatic const ref void	Synchronization semaphore mailbox	Assertions assert assume cover property sequence intersect first_match throughout within	Hierarchy package import export bind	DPI export import context pure	
							Program		
							Verilog-2001		
ANSI C style ports generate localparam constant functions			standard file I/O \$value\$plusargs `ifdef `elsif `line @*			** (power operator) configurations memory part selects variable part select			
							multi dimensional arrays signed types automatic		
							Verilog-1995		
modules parameters function/tasks always @ assign		\$finish \$fopen \$fclose \$display \$write \$monitor `define `ifdef `else `include `timescale		initial disable events wait # @ fork-join		wire reg integer real time packed arrays 2D memory		parameters function/tasks assign	+ = * / % >> <<

What Is a Datatype?



A datatype is a set of values (2-state or 4-state) that can be used to declare data objects or to define user-defined data types.

Newly introduced SV 2-state datatypes

- The Verilog datatypes have 4-state values: (0, 1, Z, X).
- SystemVerilog adds 2-state value types based on `bit`:
 - Has values 0 and 1 only.
 - Direct replacements for `reg`, `logic` or `integer`.
 - Greater efficiency at higher-abstraction level modeling (RTL).

Type	Description	Sign
<code>bit</code>	Single bit, Scalable to vector	Default unsigned
<code>byte</code>	8-bit vector or ASCII character	Default signed
<code>shortint</code>	16-bit vector	Default signed
<code>int</code>	32-bit vector	Default signed
<code>longint</code>	64-bit vector	Default signed

What Is a 'logic'?



The keyword `logic` defines that the variable or net is a 4-state data type.

- The Verilog `reg` data type is potentially confusing:
 - It can synthesize to a net or register depending on the usage.
- SystemVerilog redefines register data types as **variable** (`var`).
- SystemVerilog also defines the 4-state type as `logic`.
 - `var logic` (**variable of logic type**) replaces `reg`.
 - The `var` keyword is optional.

Verilog

```
reg clock;  
reg [15:0] b_reg;  
reg [7:0] mem8x32 [0:31];
```

SystemVerilog

```
logic clock;  
logic [15:0] b_reg;  
logic [7:0] mem8x32 [0:31];
```

Example: Usage of 2-State and 4-State Types

```
bit    [7:0] twostate;  
logic  [7:0] fourstate;  
...  
twostate <= fourstate;
```

- 2-state variables initialize to 0
 - Hides failure to initialize design

- 4-state variables initialize to X
 - Exposes failure to initialize design

4-state to 2-state

0	0
1	1
Z	0
X	0



Avoid mixing 2-state
and 4-state types

Usage of 2-state
types in TestBench

```
bit [7:0] test_number;  
bit [7:0] mem8x32 [0:31];  
int i;  
...  
mem8x32[0] = test_number;  
mem8x32[1] = i;
```

Verilog Data Type Rules: Example

```
module mone(output reg y_out,  
            input wire a_in, b_in);  
    always @(a_in or b_in)  
        y_out = a_in && b_in;  
endmodule
```

y_out is **reg** in mone –
assigned in a procedural block

a_in, b_in are **wire** in mone
and mtwo – module inputs

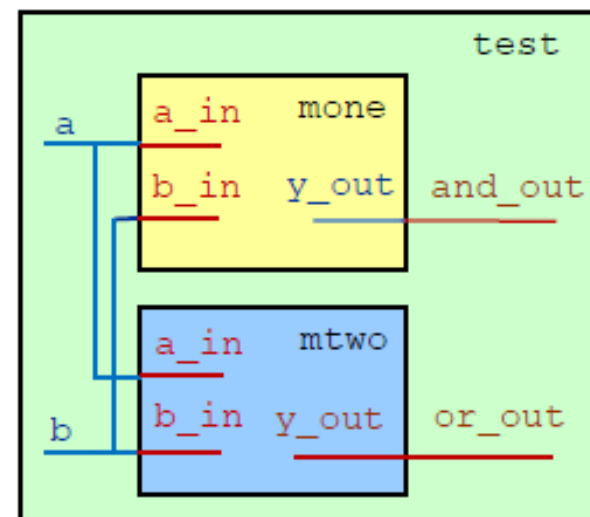
```
module mtwo(output wire y_out,  
            input wire a_in, b_in);  
    assign y_out = a_in || b_in;  
endmodule
```

y_out is **wire** in mtwo – assigned
by a continuous assignment

```
module test;  
    reg a, b;  
    wire and_out, or_out;  
    mone u1 (and_out, a, b);  
    mtwo u2 (or_out, a, b);  
    initial begin  
        a = 0;  
        b = 0;  
        ...  
    end  
endmodule
```

and_out, or_out are
wire in module test
– instance outputs

a, b are **reg** in the module test –
assigned in an initial procedural
block



Relaxed Data Type Rules: Example

```
module mone(output logic y_out,  
            input logic a_in, b_in);  
    always @(a_in or b_in)  
        y_out = a_in && b_in;  
endmodule
```

y_out is **logic** in mone –
assigned in a procedural block

a_in, b_in are **logic** in mone and
mtwo – connected to single module inputs

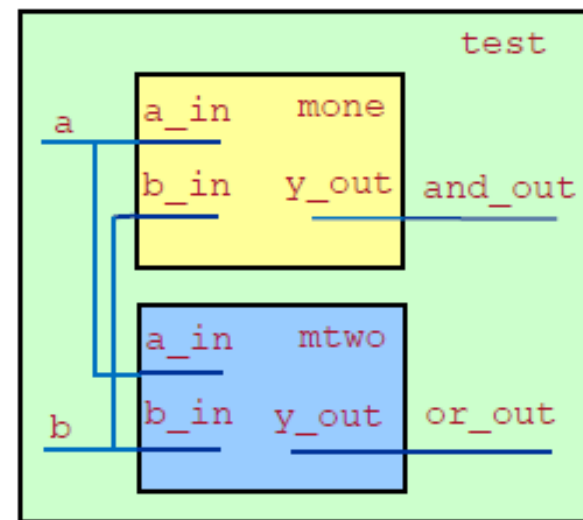
```
module mtwo(output logic y_out,  
            input logic a_in, b_in);  
    assign y_out = a_in || b_in;  
endmodule
```

y_out is **logic** in mtwo – assigned
by a single continuous assignment

```
module test;  
    logic a, b;  
    logic and_out, or_out;  
    mone u1 (and_out, a, b);  
    mtwo u2 (or_out, a, b);  
    initial begin  
        a = 0;  
        b = 0;  
        ...  
    end  
endmodule
```

and_out, or_out are
logic in module test
– single instance outputs

a, b are **logic** in the module test –
assigned in an initial procedural block



Assignments to a Variable from Multiple Procedures



Verilog allows assignments to a variable from multiple procedures. How do you avoid this in SystemVerilog?

- Special procedural blocks (`always_ff`, `always_comb`, `always_latch`) allow only a single driver to variables.
 - See the Procedures and Procedural Statements module.

```
logic op;  
  
always @(sel, a, b)  
    if (sel)  
        op = a;  
    else  
        op = b;  
  
always @(sel2, c, d)  
    if (sel2)  
        op = c;  
    else  
        op = d;
```



```
logic op;  
  
always_comb  
    if (sel)  
        op = a;  
    else  
        op = b;  
  
always_comb  
    if (sel2)  
        op = c;  
    else  
        op = d;
```



Error

`always_comb`
allows only a
single driver on
logic op.

What Is an Unsized Literal?



An Unsized literal is a number (integer or real) with a base specifier but no size specification.

- Size and value need not fit the target.
 - Value is padded or truncated to fit the target.
- Verilog-1995 extends to 32 bits:
 - With 0 if leftmost bit is 0 or 1.
 - With leftmost bit if Z or X.
- Verilog-2001 extends to full size of expression.
- New SystemVerilog single-bit literal syntax:
 - Fills whole target with that single value.
 - Avoids issues with incorrect literal sizing.
 - Allows entire vector to be set to 1.

Verilog

```
// Sized literal
// <size>'<base><value>

logic [5:0] databus;

databus = 6'b0;    // 000000
databus = 6'b1;    // 000001
databus = 6'bz;    // zzzzzz
databus = 6'bx;    // xxxxxx
databus = 4'bx;    // 00xxxx
```

SystemVerilog

```
// Unsized literal
// '<value>

logic [5:0] databus;

databus = '0;    // 000000
databus = '1;    // 111111
databus = 'z;    // zzzzzz
databus = 'x;    // xxxxxx
```

What Are Time Literals?



Time Literals are numbers written in integer or fixed-point format, followed without a space by a time unit (`fs ps ns us ms s`).

- SystemVerilog provides a time literal construct that:
 - Explicitly specifies time values.
 - Allows more control over time literals.
- Format:
 - Integer or fixed-point value.
 - Immediately followed by time unit:
 - With no space between the value and unit.
 - Having units of (`fs ps ns us ms s`).
 - Scaled to the current time unit.
 - Rounded to the current precision.
 - Also having a `1step` delay value:
 - One simulation precision unit.

```
42ns  
3.14ps  
1step
```

Only `1step` is
allowed



Never `2step`, etc.

timeunit and timeprecision



IEEE 1800-2012 22.7

- Equivalent to ``timescale`, but without file order compilation dependency.
- `timeunit` and `timeprecision` are:
 - Declarations, not directives.
 - Placed inside the module.
 - Must be the first statements of the module.
 - Only visible in the design element where they are declared and any nested units.

```
module testbench;  
    timeunit 1ns;  
    timeprecision 100ps;  
    logic status;  
    initial begin  
        #10ns status = 1'b1;  
        #10 status = 1'b0;  
    end  
endmodule
```

```
module testbench2;  
    timeunit 1ns/100ps;  
    logic status;  
  
    initial begin  
        #10ns status = 1'b1;  
        #10 status = 1'b0;  
    end  
endmodule
```

SystemVerilog

Procedural Blocks

Repeated Block Identifier at Block End



For any named block, SystemVerilog allows you to repeat the name after the block `end`, `join`, or `join variant` keyword.

Verilog-2001 allows `begin...end` and other code blocks to be named.

SystemVerilog allows a matching name to be specified with the block `end`.

It is allowed for the following code blocks:

- `module...endmodule`
- `begin...end`
- `task...endtask`
- `function...endfunction`
- `fork...join` (all forms)
- `primitive...endprimitive`
- `interface...endinterface`

You can label *any* statement.

Verilog-1995

```
module mytest;
initial
  begin : blocka
    ...
    begin : blockb
      ...
    end
  end
endmodule
```

SystemVerilog

```
module mytest;
initial
  begin : blocka
    ...
    begin : blockb
      ...
    end : blockb
  end : blocka
endmodule : mytest
```


Block Item Declarations in Unnamed Blocks



IEEE 1800-2012 A.2.8

In Verilog-2001, only named blocks can declare local variables.

In SystemVerilog, unnamed blocks can declare local variables.

- The variable is visible in the block where it is declared and in any nested blocks.
- Hierarchical references to the variable cannot be used.

```
initial begin                // unnamed block
  integer i;                 // local declaration in unnamed block
  i = 0;
  while (i <= 10) begin      // unnamed block
    $display("i:%d", i);
    if (i < 5) begin         // unnamed block
      $display("Number is:%d", i);
    end
    i = i + 1;
  end
end
end
```



Variable Declarations in for Loop Statements

IEEE 1800-2012 12.7.1

You can declare a `for` loop variable within the `for` statement.

- Variables are visible only in the loop.
- The same identifier name can be safely used in multiple (un-nested) loops.
- Good use of 2-state data types as loop variables.

Verilog

```
// loop variable declared outside loop
integer i;
initial begin
    for (i=0; i<10; i=i+1)
        ...
    initial begin
        for (i=0; i<8; i=i+1)
            ...
    end
end
```

Two loops share
single variable

int: Good use
for 2-state types

SystemVerilog

```
initial begin
    // loop variable declared in loop
    for (int i=0; i<10; i=i+1)
        ..
end

initial begin
    for (int i=0; i<8; i=i+1)
        ...
end
```

Variable 'i' visible
to for loop only

Two loops, two
independent variables

foreach Loop



IEEE 1800-2012 12.7.3

This loop iterates over all the elements of an array.

Loop variable characteristics:

- Does not have to be declared.
- Are read only.
- Only visible inside loop.

Use multiple loop variables for multidimensional arrays.

- Equivalent to nested loops.

Useful for initializing and array processing.

Equivalent SV code

```
for (int i=7; i>=0; i=i-1)  
    intarr[i] = 7-i;
```

Iterates left bound to
right bound

```
int intarr [7:0];  
...
```

```
foreach (intarr [i])  
    intarr[i] = 7-i;
```

```
int arr2d [7:0] [2:0];  
...
```

```
foreach (arr2d [k, l])  
    arr2d[k][l] = k*l;
```

Equivalent SV code

```
for (int k=7; k>=0; k=k-1)  
    for (int l=2; l>=0; l=l-1)  
        arr2d[k][l] = k*l;
```

Two loop variables for a 2D
array creates nested loops

do ... while Loop



IEEE 1800-2012 12.7.4

- The `while` loop executes a group of statements until `expression` becomes false.
- `expression` is checked at the beginning.

Verilog

```
...  
while (enable)  
    @(posedge clk)  
    count = count + 1;  
...
```

if `enable` false on loop entry;
count not incremented



IEEE 1800-2012 12.7.5

- The `expression` is checked after statements execute.
- The statement block executes at least once.
- This makes certain loop functions easier to create.

SystemVerilog

```
...  
do  
    @(posedge clk)  
    count = count + 1;  
while (enable);  
...
```

if `enable` false on loop entry;
count incremented once

Jump Statements break and continue



IEEE 1800-2012 12.8

SystemVerilog adds the `break` and `continue` keywords to control execution of loop statements.

- `break`
 - Terminates the execution of loop immediately.
 - Usually under conditional control.
- `continue`
 - Jumps to the next iteration of a loop.
 - Usually under conditional control.
- Also used in `for`, `while` and `do-while` loops.

```
repeat (8) begin
    data = {data[6:0], data[7]};
    if (data[7])
        break;
end
...
```



```
foreach (data [i]) begin
    if (data[i])
        continue;
    count = count + 1;
end
```

Quick Reference Guide: Verilog case Statement



Synthesis Pragmas	Explanation	Example
<code>parallel_case</code>	• <code>parallel_case</code> attribute directs the synthesis tool to remove inferred priority logic. • Commonly used to remove priority encoders from gate-level implementation. • Generally discouraged.	<pre>(* synthesis parallel_case *) case (parc) 0: op = a; 1,2: op = b; 3: op = c; endcase</pre>
<code>full_case</code>	• <code>full_case</code> informs the synthesis tool to treat outputs as don't care assignments for all unspecified case choices. • It doesn't work since it looks at only case item expressions and not their associated statements. • Adding the <code>default</code> statement nullifies its usage. • Generally discouraged.	<pre>(* synthesis parallel_case, full_case *) case (fullc) 0,1: op = a; 2: op = b; default: op = c; endcase</pre>

SystemVerilog priority case

 IEEE 1800-2012 12.5.3

- Modifier to the `case` statement
- **Synthesis:** Equivalent to the `full_case` attribute
- **Simulation:** Compile-time/Run-time violation report (probably warning) if no case item expression {can / does} match the case expression
 - Overlapping branches are permitted
- Can also apply to `casex`, `casez`

```
priority case (fullc)
  0:      op = a;
  0,1,2 : op = b;
  3:      op = c;
endcase
```

```
priority case (1'b1)
  en_a : op = a;
  en_b : op = b;
  en_c : op = c;
endcase
```



Runtime
warning if
one enable
not active

```
priority casez (pri)
  3'b1??: op = a;
  3'b01?: op = b;
  3'b001: op = c;
endcase
```



Runtime
warning if
pri = 0 or
pri = X

SystemVerilog unique case



IEEE 1800-2012 12.5.3

- Modifier to the `case` statement.
- **Synthesis:** Equivalent to the `full_case` and `parallel_case` attributes for synthesis.
- **Simulation:** Compile-time/Run-time violation report (probably warning) if no case item expression {can / does} match the case expression or multiple case item expressions {can / do} match the case expression.
- Defines `case` branches as complete and mutually exclusive.
 - One branch must be executed.
 - Runtime warning if no branch found for a case value.
 - No overlapping allowed.
 - Runtime warning if multiple branches found for a case value.
- Can also apply to `casex`, `casez`.
 - Beware: more likely to be overlapping.

```
unique case (fullc)
  0:      op = a;
  0,1,2 : op = b;
  3:      op = c;
endcase
```



Runtime warning
when fullc=0

```
unique case (1'b1)
  en_a : op = a;
  en_b : op = b;
  en_c : op = c;
endcase
```



Runtime warning
if multiple enables active

```
unique casez (pri)
  3'b1??: op = a;
  3'b?1?: op = b;
  3'b???1: op = c;
endcase
```



Runtime warning
if more than one bit=1

Conditional Event Control with `iff` Qualifier



IEEE 1800-2012 A.6.5

The `iff` keyword qualifies a procedural event control.

The event expression triggers only if the `iff` condition is true.

- For example, the block executes when `avec` changes and `enable` is 1.
- Block not triggered when `enable` changes.

`iff` has precedence over `or`:

- Can add parenthesis for clarity.

Limited use in RTL code.

- Latches and gated clocks.

More useful for embedded events in verification stimulus code.

Creates Latch in Synthesis

```
always @(avec iff enable == 1)
    y <= avec;
```

Hardware Equivalent

```
always @(avec)
    if (enable)
        y <= avec;
```

Typical RTL Usage

```
always @(posedge clk iff (gate == 1)
        or posedge rst)
    if (rst)
        data_out <= 8'h00;
    else
        data_out <= bvec;
```

Typical TB Usage

```
...
foreach (payload [i])
    // send payload when not suspended
    @(negedge clk iff (!suspend))
        data_out <= payload[i];
...
```

Comparing Constructs `always @*` and `always_comb`



`always @*`

- Verilog-2001 event control feature

- Can include timing and additional event controls.
- Can assign to variables which are assigned elsewhere.
- Triggered at time 0 with other blocks only if event on sensitivity list.
- Only sensitive to changes in the arguments of a called function.

```
always @*  
  if (sel == 1)  
    op = a;  
  else  
    op = b;
```



`always_comb`

- SystemVerilog procedural block

- Cannot contain any timing or event controls.
- Cannot assign to variables which are assigned elsewhere.
- Automatically triggered at time 0 after `always` and `initial` blocks.
- Sensitive to changes in any input to a called function.

```
always_comb  
  if (sel == 1)  
    op = a;  
  else  
    op = b;
```

Register Blocks with `always_ff`



IEEE 1800-2012 9.2.2.4

SystemVerilog adds a specialized procedural block for modeling registered logic:

- Variables assigned in `always_ff` cannot be assigned by another procedure.
- Contains one and only one event control.
- Cannot contain any block timing.
- Tools may issue warnings if the block does not infer registered logic.

```
always_ff @(posedge clk or posedge rst)
  if (rst)
    op <= 1'b1;
  else
    op <= ip;
```


What Are Assignment Operators?



Operators that join an operation along with a blocking assignment to the first operand of the operator.

```
initial begin
  a = 1;
  b = 2;
  a += b; // same as a = a + b, so, a = 3
  ...
```

Symbol	Usage	Meaning	Description
+=	a += b	a = a + b	add
-=	a -= b	a = a - b	subtract
*=	a *= b	a = a * b	multiply
/=	a /= b	a = a / b	divide
%=	a %= b	a = a % b	modulus
&=	a &= b	a = a & b	logical and
=	a = b	a = a b	logical or
^=	a ^= b	a = a ^ b	logical xor
<<=	a <<= b	a = a << b	left shift logical
>>=	a >>= b	a = a >> b	right shift logical
>>>=	a >>>= b	a = a >>> b	right shift arithmetic
<<<=	a <<<= b	a = a <<< b	left shift arithmetic

Pre- and Post-Increment/Decrement Operators



IEEE 1800-2012 11.4.2

Statement

```
--total;
```

These combine a blocking assignment and an increment or decrement operator:

- Pre-form (**++a**, **--a**) adds or subtracts and then uses new value.
- Post-form (**a++**, **a--**) uses a value and then adds or subtracts.
- Operators can be used as:
 - A standalone statement.
 - Part of an expression.

As Statement

```
for (int i=0; i < 7; i++)  
    ...
```

As Expression

```
initial begin  
    b = 1;  
    a = b++; // post a=1, b=2  
    a = ++b; // pre  a=3, b=3  
    a = b--; // post a=3, b=2  
    a = --b; // pre  a=1, b=1  
end
```

Wildcard Equivalence Operator



IEEE 1800-2012 11.4.6

- Allows bits to be defined as a “wildcard”:
 - Don't-care – like `casex`
 - An `x`, `z` or `?` in the *right* operand matches any value in the left
 - Asymmetric – only right side can have wildcard bits
 - Also wild inequality (`!=?`)
- Remember that Verilog has two (in)equality operators:
 - Logical (in)equality (`==`)
 - Identity (in)equality (`===`)

```
a = 4'b0101;
b = 4'b01XZ;

if (a == b)           // unknown
    ...

if (a === b)          // false
    ...

if (a ==? b)           // true
    ...

if (a !=? b)           // false
    ...

if (a ==? 4'b?1?1)    // true
    ...
```


Set Membership Operator



IEEE 1800-2012 11.4.13

- `inside` does a comparison:
 - True if expression value is contained within a value list.
 - List values can be variables (including arrays), ranges.
 - List values can have wildcards.
 - List values can overlap.
- For nonintegral expressions, it uses the equality operator (`==`):
 - Result is `1'b0`, `1'b1`
- For integral expressions, it uses a wild equality operator (`==?`):
 - Result is `1'b0`, `1'b1`, `1'bX`

```
if ( a inside {2'b01, 2'b10} )
```

```
...  
// equivalent to:
```

```
if ((a==2'b01) || (a==2'b10))
```

```
int bar [1:0] = '{5,6};  
if ( a inside {bar, [0:2]} )
```

```
...  
// equivalent to:
```

```
if (a inside {bar[1], bar[0], 0, 1, 2})
```

```
if ( a inside {2'b0?} )
```

```
...  
// equivalent to
```

```
if ( a inside {2'b00, 2'b01, 2'b0X, 2'b0Z})
```

Example: Using `inside` Operator with the `case` Statement

- You may want to execute statements when a variable or expression is within a range of values.
- `casez` can be used if range limits are on bit boundaries.
- `casez` is not useful if the range is not related to bit positions.
- For arbitrary range limits, the `inside` operator should be used.

```
bit[2:0] a;  
  
casez (a)  
  3'b00?:$display("0 or 1");  
  3'b0??:$display("2 or 3");  
endcase  
...
```

```
bit[3:0] a;  
  
case(a) inside  
  0,2      : $display("0 or 2");  
  [3:5]    : $display("3 or 4 or 5");  
  1,[6:7]: $display("1 or 6 or 7");  
  default: $display("value>7");  
endcase  
...
```

Non bit boundary range

Can mix range
and discrete values

Array Assignment Patterns



IEEE 1800-2012 10.9.1

- Define a list of values for an assignment to:
 - Array elements
 - Structure fields (see later)
- It is equivalent to individual assignments:
 - Un-sized literals are allowed
- Pattern “keys” can be used:
 - Array element index
 - `default` keyword
- There must be a pattern value for every array element:
 - Either explicitly or using `default`

```
int arr1 [3:0];  
arr1 = '{0,1,2,3};  
/* equivalent to  
arr1[3] = 0;  
arr1[2] = 1;  
arr1[1] = 2;  
arr1[0] = 3;  
*/
```

```
arr1 = '{3:1, default:0};  
/* equivalent to  
arr1[3] = 1;  
arr1[2] = 0;  
arr1[1] = 0;  
arr1[0] = 0;  
*/
```

```
// initialize mem to 0  
reg [15:0] mem [0:1023] = '{default:0};
```

SystemVerilog

User Defined Data Types

Primitive Types and User-Defined Types



- **Primitive Data Types**: Built-in types not constructed from other data types.
- **User-Defined Types**: SystemVerilog's data types can be extended with user-defined types using `typedef`.

- User-defined data types are:
 - Types named with `typedef`.
 - Types constructed from other types, such as vectors and arrays.
 - Enumerate (enum) types for user-defined value sets.
 - Structure (struct) types to bundle multiple variables into one object.
 - `union` types to store different data types in the same space.

Primitive Data Types Usage

```
logic [7:0] vec8_t;  
bit      data;  
int      digit;
```

`typedef` is used to give a user-defined name to existing 'logic' data type

```
typedef logic [7:0] vec8_t;  
...  
vec8_t vec_1, vec_2;
```

The named data type can be used now to declare other variables

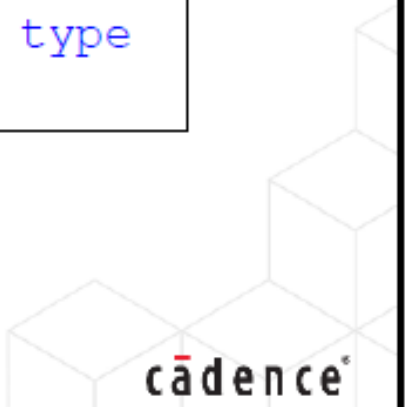
User-Defined Types: Examples

- You can declare and name a type using `typedef`:
 - Verilog standard types
 - SystemVerilog user-defined types
- The type name can be used anywhere, the type declaration is legal:
 - For example: variable or port declaration
- You must declare `typedef` before the name is used.
- The declaration must be visible in the scope where the name is used.
- An un-named type is known as an anonymous type.

```
typedef logic [7:0] vec8_t;  
  
input vec8_t op1;  
vec8_t busa, busb;  
function vec8_t checksum;  
    ...
```

```
// typed (named) type  
typedef logic [15:0] vec16_t;  
vec16_t wordvar;
```

```
// anonymous (un-named) type  
logic [16:0] wordvar;
```



What Is an Enumerated Type?



An enumerated type is a set of integral named constants.

- It is a type with user-defined values:
 - Value names must be unique in the current scope.
- Declared with the enum keyword:
 - Named or anonymous.
- Enumerated types are strongly typed.
- Should only be directly assigned:
 - A named value of the type.
 - A variable of the same type.
- Naming the type (typedef) permits typecasting:
 - `type_name' (value)`

Not Legal Direct Assignment
of Integral value
to enum variable is not allowed

```
// named enum type
typedef enum {idle, start,
              pause, done} state_t;
state_t state, next_state;

// anonymous enum type
enum {idle, start,
      pause, done} astate;

...
state = idle;
state = next_state;
state = 2'b00;
state = 2;
state = state_t'(2);
...
```

Legal

Legal

Enumerated Type Values

- Default base type of an `enum` is `int`.
- Default values increment from zero in declaration order.
- An enumerate variable can hold any value from the base type:
 - Even if there is no matching enumerate value name.
 - Casting does not check value.
- An enumerated type is the base type with mnemonics for specific values.
- An enumerate variable in an expression is replaced by its base type value.

```
typedef enum {idle, start,  
              pause, done} state_t;  
// idle=0; start=1  
// pause=2; done=3
```

0	1	2	3	4	5	6	7	8	9	...
idle	start	pause	done							

```
state_t state;  
int aint;  
initial begin  
    state = state_t'(2); // pause  
    state = state_t'(8); // no name  
  
    aint = state * 2;    // 16  
    ...
```

How to Assign Enumerate Value Encodings Explicitly



Explicit encoding can be included in an enumerated type declaration, either typed or anonymous which allows one-hot, grey or other encodings to be defined for enumerated values.

1. You can define explicit encoding for values:
 - Allows one-hot coding, etc.
2. You can mix explicit and implicit value encoding:
 - Implicit values increment from previous explicit value.
3. Value encodings must be unique:
 - Compilation error is reported if encodings overlap.

```
typedef enum {idle   = 1,  
              start  = 2,  
              pause  = 4,  
              done   = 8} state_t;
```

0	1	2	3	4	5	6	7	8	9	...
	idle	start		pause				done		

```
enum {S0=0, S1, S2=6, S3} encs;
```

0	1	2	3	4	5	6	7	8	9	...
S0	S1					S2	S3			



Error

```
enum {P0=0, P1, P2=1, P3} bad_encs;
```

P2 duplicates P1 encoding

Enumerated Type Value Name Sequences

- You can use a range notation for simple value name sequences.
 - You can mix ranged and unranged values.

```
typedef enum {S[2]} seqa_t;
```

Equivalent to

```
typedef enum {S0, S1} seqa_t;
```

```
typedef enum {go, R[3:5], stop} seqb_t;
```

Equivalent to

```
typedef enum {go, R3, R4, R5, stop} seqb_t;
```

Assigning Enumerate Base Type Explicitly



IEEE 1800-2012 6.19

- You can specify the enumeration base type:
 - Base type int by default
- This allows you to constrain the enumerate value range.
- The default value encoding will match the specified base type.
- Explicit value encoding must match type and (if explicit) length.



An explicit base type makes enumerates easier and safer to use.

Enum Declaration with **explicit base type** and **explicit encoding**

```
typedef enum bit[1:0]
{idle, start, pause, done} base_t;
// idle  = 2'b00, start = 2'b01
// pause = 2'b10, done  = 2'b11
```

00	01	10	11
idle	start	pause	done

Enum Declaration with **explicit base type** and **implicit encoding**

```
typedef enum bit[2:0]
{idle  = 3'b000,
 start = 3'b001,
 pause = 3'b010,
 done  = 3'b100} onehot0_t;
onehot0_t state;
...
state <= onehot0_t'(3'b001);
```

Enumerated Base Type vs. Enum Initial Value

- The initial value of an `enum` variable depends on its base type:
 - 0 for default `int` type and any explicit 2-state type.
 - X for an explicit 4-state type.
- You can explicitly encode a value with X:
 - Typically, to indicate an illegal value.

```
enum bit[2:0]{BONE, BTWO} var_b;
```

```
initial begin  
  $display("value %b", var_b);  
  $display("name %s", var_b.name());  
end
```

value 000
name BONE

```
enum logic[2:0] {LONE, LTWO} var_1;
```

```
initial begin  
  $display("value %b", var_1);  
  $display("name %s", var_1.name());  
end
```

value xxx
name

Empty
string

```
enum logic[2:0] {LX=3'bX, LONE=3'b001, ...} var_1;
```

```
initial begin  
  $display("value %b", var_1);  
  $display("name %s", var_1.name());  
end
```

value xxx
name LX

What Is a Structure?



A `struct` represents a collection of data types that can be referenced as a whole, or the individual data types that make up the structure can be referenced by name.

- Declare using the `struct` keyword.
- Use the “dot” notation to access individual fields.
- You can make pattern assignments to structures:
 - Ordered or keyed:
 - Not both in the same pattern.
 - Keys can be by name, type, default or a mix of these.
- You can declare arrays of structures.
- Structures can be nested.

```
typedef struct {  
    logic      id, par;  
    logic[3:0] addr;  
    logic[7:0] data;  
} frame_t;  
  
frame_t f1, two_frame[1:0];  
logic[7:0] data_in;  
...  
// individual field access  
f1.id = 1'b1;  
data_in = f1.data;  
  
// ordered assignment pattern  
f1 = '{1'b1, 1'b1, 4'h0, 8'hff};  
  
// named assignment pattern  
f1 = '{id:0, par:1, addr:0, data:0};  
  
// nested ordered assignment pattern  
two_frame = '{ '{0,0,0,255}, '{1,1,1,0}};
```


How to Assign Values to Structure Elements



Structure can use either ordered or keyed assignment patterns for assigning values to structure elements.

Order of precedence:

1. Name

- Assign fields by name.

2. Type

- Assign remaining fields of a specific type.

3. Default

- Assign remaining fields using default.
- Value must be type compatible with all remaining fields.

All fields must have an assignment.

```
typedef struct {  
    logic      id, par;  
    logic[3:0] addr;  
    logic[7:0] data;  
} frame_t;  
frame_t f1;  
  
// assignment by order  
f1 = '{1'b0, 1'b0, 12, 8'hff};  
  
// assignment by name  
f1 = '{data:8'hff, addr:12, par:0, id:1};  
  
// assignment by name and type  
f1 = '{data:8'hff, addr:12, logic:0};  
  
// assignment by name, type & default  
f1 = '{data:8'hff, logic:1, default:0};
```


What Is a Packed Structure?

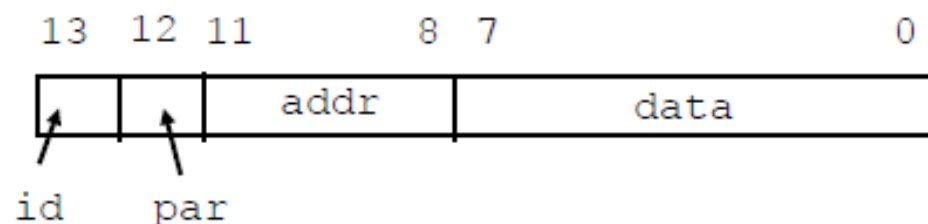


Packed Structure fields are packed together in memory without gaps and can be used as a whole with arithmetic and logical operators.

- Structures can be packed.
- All fields must be “packable”:
 - Integral values only.
 - Any field that is array or structure must also be packed.
- A packed structure can be treated as a one-dimensional vector.
 - Can still use “dot” notation to access individual structure fields.
- If any field is 4-state, then the whole structure is stored as 4-state.
 - Conversions are done automatically upon read/write.



Avoid mixing 2-state and 4-state types



```
typedef struct packed {  
    logic    id, par;  
    logic[3:0] addr;  
    logic[7:0] data;  
} frame_t;  
  
frame_t f1, f2;  
  
logic [3:0] addrbuf;  
initial begin  
    addrbuf = f1.addr;  
    addrbuf = f1[11:8];  
    f2 = f1 >> 2;  
  
    f1 = 8'haa; //14'h00aa  
    f1 = 16'h5555; //14'h1555  
    ...  
end
```

All fields are
4-state types

Same field

Padding

Truncation

New Type Declaration Regions



IEEE 1800-2012 26.0

- You can declare ports of user-defined types, but you must declare types before you use them:
 - How do modules see the type definition?
- SystemVerilog has two new name spaces:
 - Packages
 - Compilation Unit Scope (CUS):
 - To be avoided (see later for details)
- Both regions are described in more detail in a later module...

```
package mytypes;  
    typedef enum {start,done} mode_t;  
    ...  
endpackage : mytypes
```

Package declaration

```
module mtwo import mytypes::*; (  
    input logic [7:0] out,  
    output mode_t mode);  
    ...
```

Package imported
in module header

```
typedef enum {start, done} mode_t;  
  
module mone(  
    input mode_t mode,  
    output logic [7:0] out);  
    ...
```

Declaration
in CUS



Not Recommended

Quick Reference Guide: Verilog Port Connections



Ordered Port Connections	Ordered port connections, which have the risk of incorrectly ordered connections	Ordered <pre data-bbox="1493 379 2354 434">counter c1 (clk, rst, ld, data, cnt);</pre> Badly ordered <pre data-bbox="1493 548 2354 602">counter c2 (clk, ld, rst, data, cnt);</pre>
Named Port Connections	Named port connections, which are safer but very verbose	<pre data-bbox="1462 725 2448 825">counter c3 (.clk(clk), .rst(rst), .ld(ld), .cnt(cnt), .data(data));</pre>

SystemVerilog adds two options to simplify port connections:

- `.name` (dot-name)
- `.*` (dot-star)

```
module counter (  
    input  logic clk, rst, ld,  
    input  logic [7:0] data,  
    output logic [7:0] cnt);  
    ...  
endmodule
```

Rules for Using .name and .*

- In an instantiation:
 - Ordered connections with .name or .* cannot be mixed.
 - .name and .* connections cannot be mixed.
 - Named and .name connections can be mixed.
 - Named and .* connections can be mixed.
- Named connections are required for:
 - Port/signal name mismatches.
 - Port/signal width mismatches.
 - Unconnected ports.

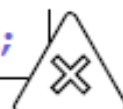
Ordered and dot-star

```
count c10 (clock, reset, .*);
```



Dot-name and dot-star

```
count c11 (.clk, .rst, .*);
```



Dot-name and named

```
count c12 (.data, .clk, .cnt, .val,  
          .rst(reset), .ld(load));
```

Dot-star and named

```
count c13 (  
    .*,  
    .ld(load),  
    .data(data[15:8]),  
    .val()  
);
```

2. Packages

- New design element similar to a module:
 - Must be compiled separately.
 - Must be compiled before elements that reference the package.
- They contain declarations to be shared between elements:
 - Types, variables, subroutines...
- You can `import` package declarations:
 - Explicit – specifically named.
 - Implicit – all using wildcard (*).
- Or directly access a declaration using the resolution operator (`::`):
 - Does not require import.

Package

```
package mytypes;  
  typedef enum {start,done} mode_t;  
endpackage : mytypes
```

Explicit import into CUS

```
import mytypes::mode_t;  
module mone(input mode_t mode...);  
  ...
```

Wildcard import

```
module mone(...);  
  import mytypes::*;  
  
  mode_t mode,  
  ...
```

Resolved name

```
module mone(...);  
  
  mytypes::mode_t mode,  
  ...
```


What Is an Explicit Import?



An explicit import only imports the symbols specifically referenced by the `import`.

- Explicit import directly loads declaration into the module.
- Declaration must be unique in the current scope:
 - Compilation error if local or other explicitly imported declarations have same name.
- Only the symbols specifically referenced are imported.
- An explicit import in a CUS follows compilation unit rules:
 - Local declarations override CUS declaration.
 - Even for an explicit import.
 - Compilation Unit Scope should be avoided if possible.

```
package P1;  
  localparam int c1 = 10;  
  typedef enum {start,stop} mode_t;  
endpackage : P1
```

```
module mone(...);  
  import P1::c1;  
  import P1::mode_t;  
  
  logic [7:0] c1;  
  mode_t mode;  
  ...  
  if (mode==stop)  
  ...
```

Local c1 clashes
with P1::c1



Unknown identifier
(use P1::stop)



```
import P1::c1;  
module mone(...)  
  logic [7:0] c1;  
  ...
```

CUS import

Local c1 takes
precedence

What Is a Wildcard Import?



A wildcard import allows all identifiers declared within a package to be imported provided the identifier is not otherwise defined in the importing scope.

Wildcard import makes declaration candidate for import:

- Local or explicitly imported declarations can override wildcard imported declarations.
- Package declarations still visible through resolved name.

```
package P1;  
  localparam int c1 = 10;  
  typedef enum {start,stop} mode_t;  
endpackage : P1
```

```
module mone(...);  
  import P1::*;  
  logic [7:0] c1;  
  mode_t mode;  
  ...  
  if (mode==stop)  
  ...  
  $display("%h", P1::c1);  
  ...  
endmodule
```

Local c1
overrides
P1::c1

All package
declarations
visible

Resolved name to access
package declaration



Local or explicitly imported
declarations take precedence
over wildcard imports.

Import and Export Statements

- The import is a statement that is equivalent to a data declaration.
- Declarations are only visible from the import onwards.
- Fewer visibility issues.
- Better readability.
- Imports do not chain:
 - Importing P1 into P2, and then P2 into mone does not make P1 declarations visible in mone.
 - P1 must be separately imported into mone...
 - ...or add an export to P2 to make P1 visible as part of P2.
 - export P1::*;
 - Rarely used.



Place imports at the beginning of the design element.

```
package P1;  
  int c1 = 5;  
  ...  
endpackage : P1
```

```
package P2;  
  import P1::*;  
  int c2 = c1;  
  ...  
endpackage : P2
```

Could add
export P1::*;

```
module mone(...);  
  
  logic [7:0] d = c2;  
  
  import P2::*;  
  
  initial begin  
    $display("%0d", c2);  
  
    $display("%0d", c1);  
    ...  
  end
```

Error –
c2 undefined

Error –
c1 undefined

Ambiguity and Resolved Names



How to resolve ambiguous references due to: “Multiple Declarations in separate packages using same name.”

Solutions

- Try to avoid duplicate names
- Structure packages carefully
 - Only required declarations visible
- Use resolved names
- Use explicit imports

```
package P1;  
    typedef enum {start,done} mode_t;  
    int c1 = 5;  
    ...  
endpackage : P1
```

```
package P2;  
    int c1 = 8;  
    ...  
endpackage : P2
```

Solution 1

```
module mone(...);  
    import P1::*, P2::*;  
  
    logic [7:0] d = c1;  
    logic [7:0] e = P2::c1;  
    ...
```

Error –
ambiguous



OK – resolved

Solution 2

```
module mone(...);  
    import P1::*, P2::*;  
    import P2::c1;  
  
    logic [7:0] d = c1;  
    ...
```

Explicitly
importing c1 from
package you want
to always infer

OK – resolved

SystemVerilog

Arrays

Arrays



IEEE 1800-2012 7.4

- SystemVerilog allows arrays of any type including:
 - Events, structures, and enumerates
- Standard, Verilog-like arrays are defined as unpacked:
 - Index declared after name
 - Elements stored separately

Assignment patterns can be used to assign unpacked arrays.

```
int inta [2:0][1:0];
```

2D array
of int

```
typedef enum bit [1:0]  
    {S0, S1, S2, S3} seq_t;  
seq_t seq5 [4:0];
```

enum array

```
logic[7:0] unpackarr [1:0];
```

Unpacked array of bytes

7 0

logic[7:0][1]

unpackarr[1]

7 0

logic[7:0][0]

unpackarr[0]

array assignments

```
unpackarr = '{8'h00, 8'hff};  
unpackarr = '{default:8'h00};
```

Unpacked Multidimensional Arrays



The term unpacked array is used to refer to the dimensions declared after the data identifier name.

- Unpacked dimensions are stored separately.
- You can read and write any slice from one element to a whole array:
 - Must match size, layout and type.
 - Indexing precedence: left to right.
- Assignment patterns are useful for loading unpacked arrays:
 - Keyed or ordered.
 - Can be nested.
 - Can also contain expressions, for example: repetition.

```
int inta [2:0][0:1];  
int intb [1:3][2:1];  
int intc [15:0][0:255];
```

3x2 arrays
of int

```
...
```

```
inta = intb;
```

Both 3x2 arrays of int

```
inta[2] = intb[3];
```

Both 2x arrays of int

```
inta = '{ {2,1}, {5,4}, {3,2} }';
```

```
// inta[2][0] = 2
```

```
// inta[2][1] = 1
```

```
// inta[1][0] = 5
```

```
...
```

```
intb = '{ 3 {1,2} }';
```

```
// intb[1][2] = 1
```

```
// intb[1][1] = 2
```

```
// intb[2][2] = 1
```

```
...
```

```
intc = '{ default: 100 }';
```

```
// intc[0][0] = 100
```

```
...
```

Nested ordered
assignment pattern

Pattern expression

Keyed assignment

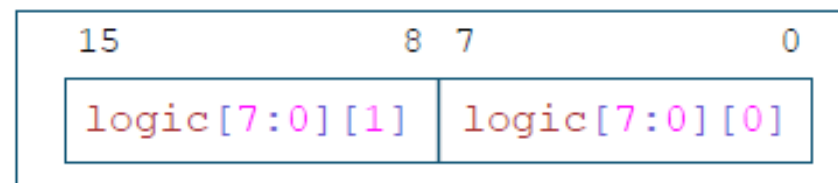
Packed Arrays



A packed array is a mechanism for subdividing a vector into subfields, which can be conveniently accessed as array elements.

- Packed arrays:
 - Index declared before name
 - Stored as a single vector
 - Can be operated upon and accessed as a single vector
 - Indexing precedence from left to right
- Only single-bit data types can be packed: `bit`, `logic`, `reg`:
 - And recursively, other packed objects of these types
- Packed arrays obey assignment rules for padding and truncation.
- A packed array is equivalent to a vector with predefined bit fields.

Packed array of bytes



```
logic[1:0] [7:0] packarr;  
logic[7:0] short = 8'hAA;  
logic[23:0] long = 24'h555555;  
...  
packarr = 16'h0;  
packarr = packarr << 1;  
packarr = ~packarr;  
  
packarr = short;    //16'h00aa  
packarr = long;     //16'h5555
```


Mixed Packed and Unpacked Arrays

- Arrays can have both packed and unpacked dimensions:
 - Like Verilog memories.
 - Any number of dimensions.
- Mixed arrays can be declared with `typedef`:
 - Unpacked dimensions placed after type name.
- Precedence for indexing is like the following:
 - Unpacked dimensions left to right...
 - ..then packed dimensions left to right.
- Generally, only fully packed arrays are synthesizable.

Verilog memory – unpacked array of packed bytes

```
reg [7:0] mem [0:255];
```

```
typedef logic [7:0] mem_t [0:255];  
mem_t mem1;
```

Indexing order

2 3 1

```
bit [3:0] [5:1] v1 [7:0];  
// v1      = array of 2D packed arrays  
// v1[3]   = 2D packed array[3:0][5:1]  
// v1[3][2] = 1D packed array[5:1]  
// v1[3][2][1] = scalar bit
```


Verilog-2001 Tasks and Functions Review



Tasks

- Any number of input, output or inout arguments – can be ANSI C style
- May contain timing: @ (...), #delay, wait

```
task write_mem (  
    input  [7:0]  waddr, wdata,  
    output [7:0]  indata, addr);  
begin  
    indata  <= wdata;  
    addr    <= waddr;  
    write   <= 1'b0;  
    #5 write <= 1'b1;  
    #10 write <= 1'b0;  
end  
endtask
```

Functions

- One or more input arguments only – can be ANSI C style
- No timing allowed – function must complete in zero time
- Returns single value – vector, real or integer

```
function [7:0] flip(  
    input [7:0] word);  
    reg [7:0] tempword;  
    integer i;  
begin  
    for (i=7; i>=0; i=i-1)  
        tempword[i] = word[7-i];  
    flip = tempword;  
end  
endfunction
```

SystemVerilog Optional begin / end and Named Ends

IEEE 1800-2012 13.3

- You can omit the `begin` and `end` keywords.
- You can repeat the subroutine name with `endtask` or `endfunction`.
 - SystemVerilog verifies that it is the same name.

```
function [7:0] flip
  (input [7:0] word);
  logic [7:0] tempword;
//begin
  for (int i=0;i<=7;++i)
    tempword[i] = word[7-i];
  flip = tempword;
//end
endfunction : flip
```

SystemVerilog

```
task write_mem (
  input  [7:0] waddr, wdata,
  output [7:0] indata, addr);
//begin
  indata <= wdata;
  addr   <= waddr;
  write  <= 1'b0;
#5 write <= 1'b1;
#10 write <= 1'b0;
//end
endtask : write_mem
```

SystemVerilog

Void Functions



IEEE 1800-2012 7.4

- You can declare functions that do not return a value.
 - Define return type as `void`.
- You call a void function as a statement:
 - All restrictions of functions still apply.
- `void` can also be used to discard a function return value:
 - By casting the return type to `void`.

```
function void printerr (input int errs);  
  if (errs == 0)  
    $display ("No Errors");  
  else  
    $display ("%0d Errors", errs);  
endfunction  
...  
initial begin  
  ...  
  printerr(status);  
  ...
```

```
function bit myfunct (...);  
  ...  
endfunction  
...  
initial begin  
  void' (myfunct (...));  
  ...
```

Function Output Arguments



IEEE 1800-2012 13.4

- You can provide `input`, `output`, and `inout` function arguments.
 - Argument default is `input` of type `logic`.
 - `output/inout` arguments let functions return multiple values.
 - `output/inout` arguments let `void` functions return values.
- Function becomes a general-purpose synthesizable subroutine.

```
function [7:0] addcarry (input [7:0] a, b,  
                        output carry);  
    {carry, addcarry} = a + b;  
endfunction
```

```
logic [7:0] a, b, sum;  
logic cry;
```

```
initial begin  
    sum = addcarry(a, b, cry);  
    ...
```

cry is an output

```
// a and b default to inputs  
function void add ( integer a, b,  
                  output integer sum);  
    sum = a + b;  
endfunction
```

```
always @(a or b)  
    // void function call  
    add (a, b, sum);
```


Default Argument Values

- You can specify default values for subroutine arguments.
 - SystemVerilog uses default values for those not passed – with rules!
 - Must have unambiguous mapping of actuals to formals.
 - It is an error for a formal parameter to not have an unambiguous value.

Error:
'j' gets value
of 2 and 'k' is not
defined

```
// default argument values in task definition
task read (int j = 0, int k, int data = 1);
. . .
endtask

int val = 21;

// invocation of task with default arguments
read ( , 5);      // equivalent to (0, 5, 1)
read (2, val);    // equivalent to (2, 21, 1)
read ( , 5, 7);   // equivalent to (0, 5, 7);
read (2);         // ERROR - k not defined
```

Explicit Return: `return`



IEEE 1800-2012 13.4.1

- Executing a `return` statement immediately exits a subroutine.
 - Allows easier structuring of subroutines
- In a function, you can include an expression to return a function value.

```
function integer mult (input integer num1, num2);  
  if ((num1 == 0) || (num2 == 0)) begin  
    $display("Zero multiply");  
    return 0;  
  end  
else  
  mult = num1*num2;  
endfunction
```

```
task printstatus (input int errs);  
  if (errs == 0) begin  
    $display("Zero Errors");  
    return;  
  end  
  else begin  
    $display("%d Errors", errs);  
    case (errs)  
      ...  
    endcase  
  end  
endtask
```

SystemVerilog

Interfaces

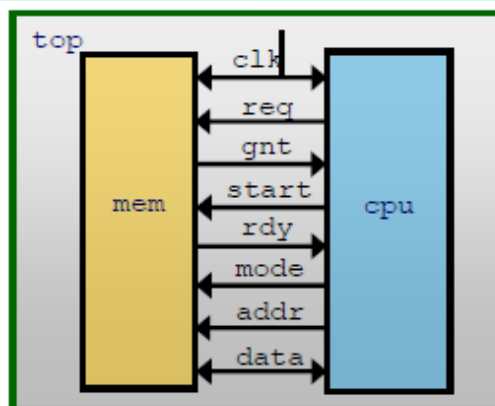
Verilog Hierarchical Connections



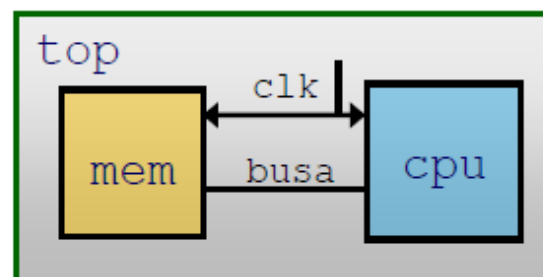
One Verilog hierarchical connection requires 5 declarations:

- Two port declarations in modules `mem` and `cpu`
- Signal declaration in `top`
- Signal added to each instantiation of `mem` and `cpu`

Problem: Creating and maintaining multiple connections is tedious.



Using Interface



```
module memory(  
    input logic clk, req, start,  
            logic [1:0] mode,  
            logic [7:0] addr,  
    inout wire [7:0] data,  
    output logic gnt, rdy);  
    ...  
endmodule
```

```
module cpucore(  
    input logic clk, gnt, rdy,  
    inout wire [7:0] data,  
    output logic req, start,  
            logic [7:0] addr,  
            logic [1:0] mode);  
    ...  
endmodule
```

```
module top;  
    logic req, gnt, start, rdy;  
    logic clk = 0;  
    logic [1:0] mode;  
    logic [7:0] addr;  
    wire [7:0] data;  
  
    memory mem(.clk, .req, .start,  
                .mode, .addr, .data, .gnt, .rdy);  
  
    cpucore cpu(.clk, .gnt, .rdy, .data,  
                .req, .start, .addr, .mode);  
    ...  
endmodule
```

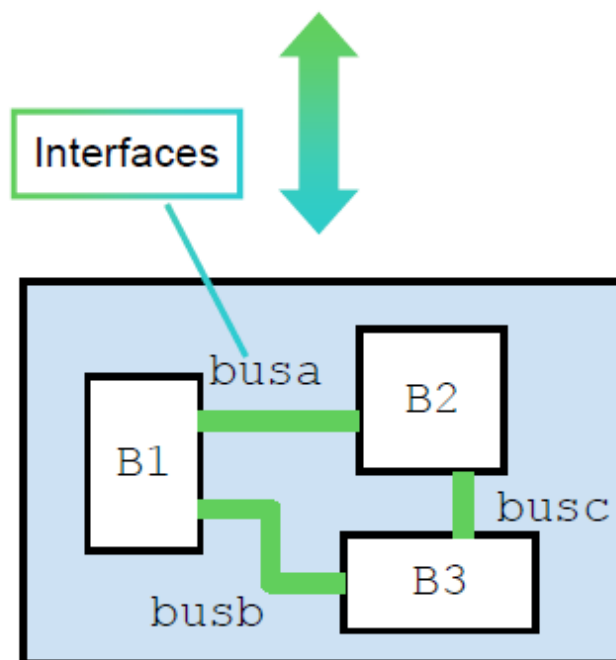
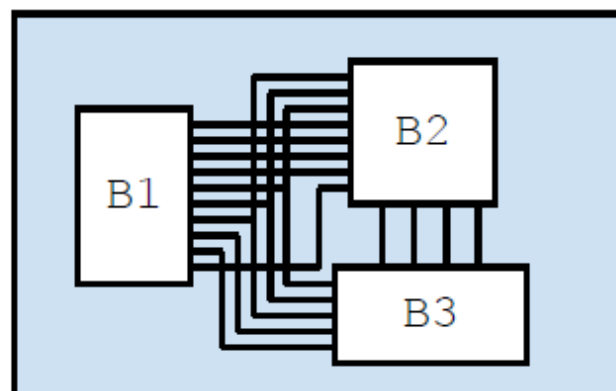
What Is an Interface?



A SystemVerilog interface is, at its most basic level, simply a bundle of external nets and/or variables normally shared by one or more modules by virtue of port connections.

SystemVerilog provides the interface construct to:

- Encapsulate communication between hardware blocks.
- Provide a mechanism for grouping together multiple signals into a single unit that can be passed around the design hierarchy.
- Enable abstraction in RTL design.



Example with Interfaces

```
interface ifa;  
    logic req, start, gnt, rdy;  
    logic [1:0] mode;  
    logic [7:0] addr;  
    wire [7:0] data;  
    ...  
endinterface : ifa
```

①

Interface
declaration

```
module memory(  
    input bit clk,  
    ifa bus);  
    ...  
endmodule
```

②

Interface used
as a directionless
port type in
module declarations

```
module cpucore(  
    input bit clk,  
    ifa bus);  
    ...  
endmodule
```

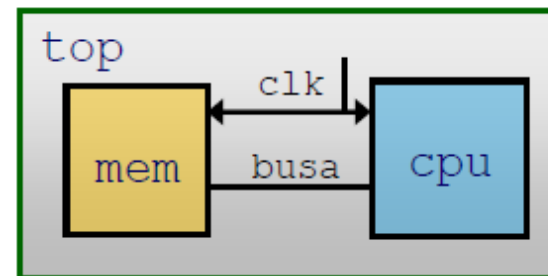
```
module top;  
    logic clk = 0;  
    ifa busa();  
    memory mem (clk, busa);  
    cpucore cpu (clk, busa);  
    ...  
endmodule
```

③

Interface
instantiation

④

Instantiation mapped
to module ports





Quick Reference Guide: More Facts on Interfaces

You declare an interface as a hierarchical object, much like a module:

- You instantiate it in a module, like any other hierarchical block.
- You use the interface like a type in port lists and port maps.

A simple interface is just a named bundle of nets or variables:

- Similar to a `struct` type.
- You can reference the nets or variables where needed.

Interfaces can also contain module-like features for defining signal relationships:

- Continuous assignments, tasks, functions, initial/always blocks, etc.
- Can further instantiate interfaces.
- Can be generated or arrayed.
- Cannot declare or instantiate module-specific items:
 - Modules, primitives, specify blocks, and configurations.



How to

Bind Interface Instance to Module Ports of Interface Type



SystemVerilog port mapping rules also apply to interfaces.

1. You can map by position or name.
2. You can use dot-name (.name) mapping if instance name matches port name.
3. You can use dot-star (.*) mapping if all names match.

```
module modone (  
    input bit clk,  
    ifa busa);  
    ...
```

```
module modtwo (  
    input bit clk,  
    ifa busb);  
    ...
```

```
module top;  
    logic clk = 0;
```

Creating Interface Instances

```
    ifa oneif(), twoif(), busa(), busb();
```

```
    modone mem_pos(clk, oneif);
```

```
    modone mem_nam(.clk(clk), .busa(twoif));
```

```
    modone mem_dotnam(.clk, .busa);
```

```
    modtwo mem_dotstr(.*);  
    ...
```

Binding Interface Instances

Positional

Named

dot-name

dot-star

How to

Access Interface Items Through Module Ports Bound to the Interface Instance



For a module with an interface *port*, you access interface items using the port name as a prefix.

```
interface ifa;
    logic req, start, gnt, rdy;
    logic [1:0] mode;
    logic [7:0] addr;
    wire  [7:0] data;
    ...
endinterface : ifa
```

```
module memory ( input bit clk, ifa bus );
    reg [31:0] mem [0:31];
    logic read, write;
    assign read  = (bus.gnt && (bus.mode == 0) );
    assign write = (bus.gnt && (bus.mode == 1) );
    always @(posedge clk)
        if (write)
            mem[bus.addr] = bus.data;
    assign bus.data = read ? mem[bus.addr] : 'z;
endmodule
```

1. Pass the Interface as port bus.

2. Access the Interface item using module port name.

How to

Access Interface Items Through the Interface Instance Identifier



For a module with an interface *instance*, you access interface items using the instance name as a prefix.

```
module top;
    logic clk;

    ifa busa();

    memory mem (clk, busa);
    cpucore cpu (clk, busa);
    ...
    always @(busa.rdy)
        if (busa.mode == 0)
            ...
endmodule : top
```

```
interface ifa;
    logic req, start, gnt, rdy;
    logic [1:0] mode;
    logic [7:0] addr;
    wire [7:0] data;
    ...
endinterface : ifa
```

1. Declare Interface instance busa.

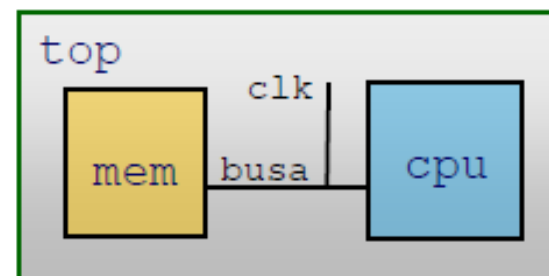
2. Access Interface signal using instance name.

How to Define Interface Ports



An interface can have its own ports:

- Connected like any module port.
- Used to share an external signal.



```
interface ifa (input clk);  
    logic req, start, gnt, rdy;  
    logic [1:0] mode;  
    logic [7:0] addr;  
    wire [7:0] data;  
endinterface : ifa
```

1. Define 'clk' port in Interface declaration.

```
module memory( ifa bus );  
    ...  
endmodule
```

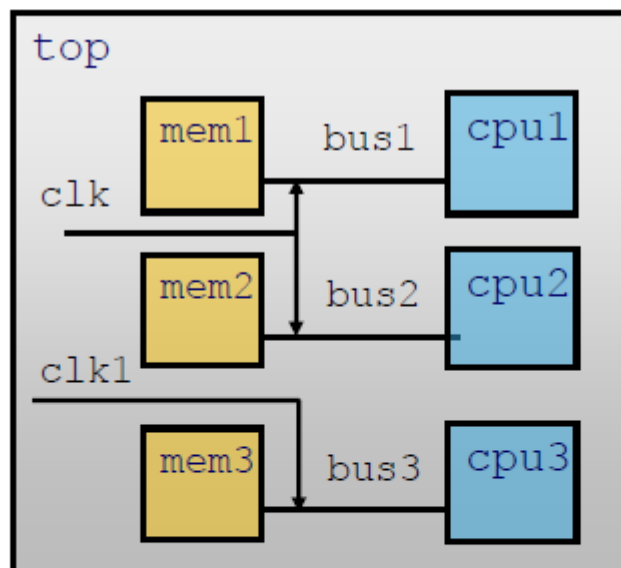
```
module cpucore( ifa bus );  
    ...  
endmodule
```

2. Connect 'clk' port of 'top' module to Interface port during instantiation.

```
module top;  
    logic clk = 0;  
  
    ifa busa(clk);  
  
    memory mem (busa);  
  
    cpucore cpu (busa);  
    ...  
endmodule
```

Example: Interface Port Applications

- Common signals for instances of different interface definitions.
- Different signals for different instances of the same interface definition.



```
interface pds_if (input clk);  
    logic [7:0] address;  
    wire [7:0] data;
```

```
interface hbus_if (input clk);  
    logic [7:0] address;  
    logic [7:0] data;
```

```
module top;  
    logic clk, clk1;  
  
    pds_if bus1(clk);  
    mem_pds mem1 (bus1);  
    cpu_pds cpu1 (bus1);  
  
    hbus_if bus2(clk);  
    mem_hbus mem2 (bus2);  
    cpu_hbus cpu2 (bus2);  
  
    hbus_if bus3(clk1);  
    mem_hbus mem3 (bus3);  
    cpu_hbus cpu3 (bus3);  
endmodule : top
```

How to Define Interface Parameters



You can parameterize interface just like a module.

1. You can define them as part of interface header.

2. You can also define them inside interface with 'parameter' type.

```
interface fastbus #(DBUS = 32, ABUS = 8) (input clk);  
    logic [DBUS-1:0] data;  
    logic [ABUS-1:0] address;  
    ...  
endinterface
```

```
interface slowbus;  
    parameter WIDTH = 16;  
    logic [WIDTH-1:0] a, b;  
    ...  
endinterface
```

```
module test;  
    fastbus #(8, 5) bus8x5(clk); // 8-bit DBUS and 5-bit ABUS  
  
    fastbus #(8) bus8x8(clk); // 8-bit DBUS and 8-bit ABUS  
  
    slowbus #(.WIDTH(8)) bus2(); // 8-bit a, b variables  
    .....
```

3. You can override parameters during interface instantiation.

What Is modport?



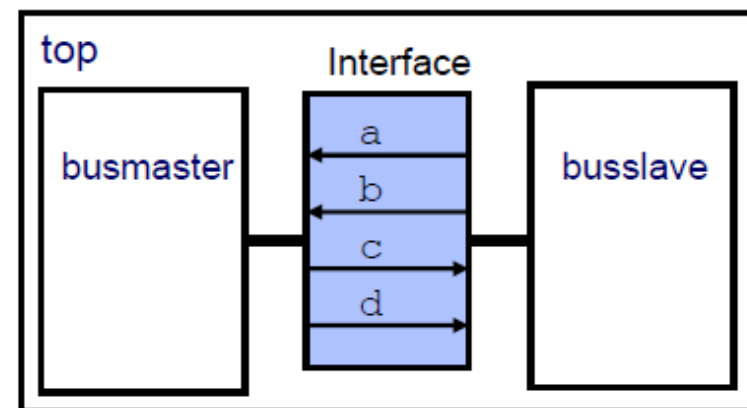
To restrict interface access within a module, there are `modport` lists with directions declared within the interface. The keyword `modport` indicates that the directions are declared as if inside the module.

Modports create different views of an interface.

- Specify a subset of interface signals accessible to a module.
- Specify direction information for those signals.

You can specify a modport view for a specific module in two ways:

1. In the module declaration.
2. In the module instantiation.



```
interface mod_if;  
  logic a, b, c, d;  
  modport master (input a,b, output c,d);  
  modport slave  (output a,b, input c,d);  
  modport subset (output a, input b);  
endinterface
```


Selecting the Interface Modport by Qualifying the Module Port Interface Type



An interface modport can be selected using the module declaration port of interface type.

1. `busmaster` declares interface port `mbus`:

- Type is `mod_if`
- Modport is `master`

2. `busslave` declares interface port `sbus`:

- Type is `mod_if`
- Modport is `slave`

3. `testbench` instantiates interface and modules as before.

```
interface mod_if;
    logic a, b, c, d;
    modport master (input a,b, output c,d);
    modport slave (output a,b, input c,d);
endinterface
```

```
module busmaster (mod_if.master mbus);
...
endmodule

module busslave (mod_if.slave sbus);
...
endmodule
```

```
module testbench;
    mod_if busa();
    busmaster M1 (.mbus(busa));
    busslave S1 (.sbus(busa));
    ...
endmodule
```

Selecting the Interface Modport by Qualifying the Module Port Interface Binding



An interface modport can be selected during the port mapping of module instantiation.

1. `busmaster` declares interface port `mbus` of type `mod_if`.
2. `busslave` declares interface port `sbus` of type `mod_if`.
3. `testbench` instantiates `mod_if` instance `busb`.
4. `testbench` maps the `mbus` port of `M1` to `modport master` of `busb`.
5. `testbench` maps the `sbus` port of `S1` to `modport slave` of `busb`.

```
interface mod_if;  
    logic a, b, c, d;  
    modport master (input a,b, output c,d);  
    modport slave  (output a,b, input c,d);  
endinterface
```

```
module busmaster (mod_if mbus);  
    ...  
endmodule  
  
module busslave (mod_if sbus);  
    ...  
endmodule
```

```
module testbench;  
    mod_if busb();  
    busmaster M1 (.mbus(busb.master));  
    busslave S1 (.sbus(busb.slave));  
    ...  
endmodule
```

What Are Interface Methods?



A sub-routine defined within an Interface is called an Interface method.

Writing/reading interface signals can be handled by tasks:

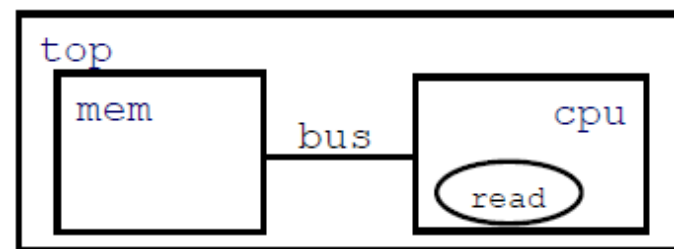
- Abstract interface communication

However, every module must contain copies of the tasks:

- Maintainability issues

Solution is to declare tasks as part of the interface:

- Accessible to any module connected to the interface



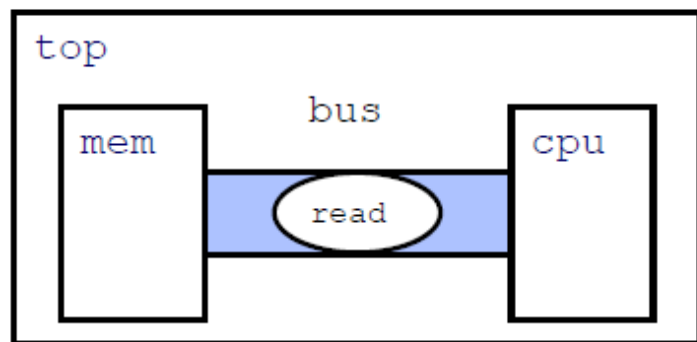
```
module cpucore(ifa bus);

    task read (input address);
        @(posedge clk);
        bus.addr = address;
        ...
    endtask
    ...
    read(8'hFF);
    ...
endmodule
```

```
interface ifa(input clk);
    logic req, start, gnt, rdy;
    logic [1:0] mode;
    logic [7:0] addr;
    wire [7:0] data;
endinterface : ifa
```

Accessing Interface Methods in Module

- Same syntax and statements as in a module task definition
- Declare once – visible to any module connected to interface
- Call methods through interface port or instance



```
module cpubcore(ifa bus);  
    ...  
    bus.read(addr,data);  
    ...  
endmodule
```

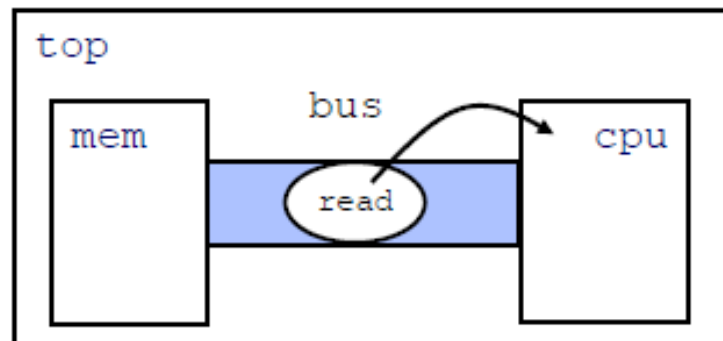
```
interface ifa (input clk);  
    logic req, start, gnt, rdy;  
    logic [1:0] mode;  
    logic [7:0] addr;  
    wire [7:0] data;  
  
    task read (input byte raddr,  
              output byte rdata);  
        @(posedge clk);  
        addr = raddr;  
        ...  
        rdata = data;  
    endtask  
endinterface : ifa
```

Controlling Access to Interface Methods Using Modports

If connected to an interface modport, a module sees only:

- Ports defined in that modport
- Methods imported from that modport

Use `import` to make methods visible via the modport



```
interface ifa (input clk);
    logic req, start, gnt, rdy;
    logic [1:0] mode;
    logic [7:0] addr, data;

    modport cif (
        input clk, gnt, rdy,
        output start, req, mode, addr,
        inout data,
        import read );

    task read (input byte raddr,
              output byte rdata);
        ...
    endtask

endinterface : ifa
```

```
module cpucore(ifa.cif bus);
    ...
    bus.read(addr, data);
    ...
endmodule
```

How to

Declare a Module Port of a Generic Interface Type



You can use the `interface` keyword rather than a specific interface type in your module declaration's list of ports. This serves as a placeholder for an interface type and you can select the interface when instantiating the module.

You can declare a module port of a generic interface type:

- This defers actual interface selection until module instantiation.
- Must use Verilog-2001 ANSI-style port declaration.

```
interface ifa;
    logic req, start, gnt, rdy;
    logic [1:0] mode;
    ...
endinterface : ifa
```

```
module memory(interface bus);
    ...
endmodule : memory
```

```
module cpucore(interface bus);
    ...
endmodule : cpucore
```

```
module top;

    ifa busa();

    memory mem (.bus(busa));
    cpucore cpu (.bus(busa));

endmodule : top
```

1. Use a generic interface port in module definition.

2. Map Specific interface during instantiations.